

DETAILNÍ TESTOVACÍ SCÉNÁŘ

Úspěšné vytvoření rezervace a její následné zrušení uživatelem

1. Účel a popis scénáře

Tento scénář podrobně popisuje proces úspěšného vytvoření rezervace prostřednictvím REST API, přičemž vstupní data plně vyhovují všem business pravidlům rezervačního systému, a jejího následného zrušení uživatelem pomocí dedikovaného endpointu.

Cílem je ověřit:

- Správnou detekci validních vstupů v metodě `ReservationService.validateReservation(...)`.
- Správné přiřazení statusu `CONFIRMED` (protože počet osob ≤ 4).
- Úspěšné uložení rezervace do repository.
- Funkčnost operace zrušení (`cancelReservation`) a přechod statusu na `CANCELLED`.

Scénář demonstuje kompletní „happy path“ tok zpracování požadavku.

2. Předpoklady

- Aplikace je spuštěna (Spring Boot context načten).
- Používá se `InMemoryRoomRepository` obsahující místnosti: Alpha (ID 1, kapacita 4), Beta (ID 2, kapacita 8), Gamma (ID 3, kapacita 12).
- `InMemoryReservationRepository` neobsahuje žádné aktivní rezervace pro testovanou místnost a časový interval (žádný konflikt).
- Přístup je realizován přes HTTP POST na endpoint `/reservations` (`ReservationController`) a následně `PATCH /{id}/cancel`.
- Testovací klient (např. Postman, curl nebo automatizovaný test) je připraven odeslat validní JSON a zpracovat odpovědi.

3. Testovací data

Následující vstupní data jsou navržena tak, aby plně vyhovovala všem business pravidlům (kapacita, pracovní doba, zarovnání, doba trvání, víkendová omezení).

Parametr	Hodnota
roomId	1 (místnost „Alpha“, kapacita = 4)
people	3 (v rámci kapacity místnosti)
start	2026-05-18 09:00:00 (pondělí, 9:00 – v pracovní době)
end	2026-05-18 10:00:00 (doba trvání 60 minut)
status	null (bude nastaven službou na <code>CONFIRMED</code>)

4. Detailní kroky provedení

Níže je popsán kompletní tok zpracování požadavku na vytvoření rezervace a následného zrušení včetně očekávaného chování jednotlivých vrstev aplikace.

#	Krok / Komponenta	Akce	Očekávaný výsledek / Poznámka
1	ReservationController	Přijme POST /reservations s validním JSON tělem	Deleguje na service.createReservation(reservation).
2	ReservationService.createReservation	Ověří non-null, načte Room (ID 1) z repository	Room nalezen; capacity = 4. Pokračuje ve zpracování.
3	ReservationService.validateReservation	Provede všechny validační kontroly (null, intervaly, alignment, people, capacity, víkend, duration, pracovní doba)	Všechny kontroly prošly úspěšně. Žádná výjimka. Metoda vrací true. (pozn.: capacity check prošel, protože $3 \leq 4$)
4	ReservationService	Zavolá hasConflict(roomId, start, end) na repository	Vrátí false (žádný konflikt). Rezervace může být vytvořena.
5	ReservationService	Určí status rezervace: if (people <= 4) → CONFIRMED else PENDING	Status = CONFIRMED ($3 \leq 4$).
6	ReservationService	Vytvoří nový Reservation objekt a zavolá repository.save(...)	Rezervace uložena do InMemory mapy, přiřazeno nové ID (např. 1). Vracena instance s ID a statusem CONFIRMED.
7	ReservationController	Vrátí vytvořenou rezervaci jako HTTP odpověď	HTTP 200 OK + JSON tělo s kompletní rezervací (včetně ID a status=CONFIRMED).
8	ReservationController	Přijme PATCH /reservations/{id}/cancel (id z kroku 7)	Deleguje na service.cancelReservation(id).
9	ReservationService.cancelReservation	Ověří id != null, načte rezervaci z repository	Rezervace nalezena; aktuální status = CONFIRMED (není CANCELLED).
10	ReservationService	Nastaví status na CANCELLED a zavolá save(...)	Stav aktualizován v repository. Rezervace zůstává v systému (pouze změna statusu).
11	ReservationController	Vrátí aktualizovanou rezervaci	HTTP 200 OK + JSON s status=CANCELLED. Operace úspěšná.

Klíčové zjištění: Vzhledem k tomu, že všechny validační kontroly v validateReservation prošly úspěšně a nebyl detekován konflikt, byla rezervace vytvořena se statusem CONFIRMED. Následné zrušení proběhlo bezchybně – status byl změněn na CANCELLED a data zůstala konzistentní v repository. Žádná výjimka nebyla vyhozena.

5. Očekávaný výsledek

Po úspěšném provedení scénáře:

- Rezervace je úspěšně vytvořena a uložena do InMemoryReservationRepository s přiřazeným ID a statusem CONFIRMED.
- Po volání cancel endpointu je status rezervace změněn na CANCELLED a změna je perzistována.
- Žádná výjimka (IllegalArgumentException) není vyhozena během validace ani zrušení.
- HTTP odpovědi: 200 OK pro oba požadavky (vytvoření i zrušení) s odpovídajícím JSON tělem.

- Stav systému zůstává konzistentní – rezervace existuje v repository po celou dobu (pouze změna statusu).
- Žádná částečná data ani nekonzistence nejsou vytvořeny.

6. Zapojené komponenty a reference na kód

Hlavní třídy a metody podílející se na zpracování:

- `ReservationController.create(...)` – přijímá POST požadavek a deleguje na službu.
- `ReservationController.cancel(...)` – přijímá PATCH požadavek na zrušení a deleguje na službu.
- `ReservationService.createReservation(Reservation)` – orchestrace vytvoření, validace, kontrola konfliktu, nastavení statusu a uložení.
- `ReservationService.cancelReservation(Long)` – načtení, kontrola stavu, změna na CANCELLED a uložení.
- `ReservationService.validateReservation(Reservation, int)` – obsahuje veškerá business pravidla.
- `InMemoryRoomRepository.findById(...)` – poskytuje data o místnosti a kapacitě.
- `InMemoryReservationRepository` – `hasConflict(...)`, `save(...)`, `findById(...)`.
- Modely: `Reservation`, `Room`, `ReservationStatus` (CONFIRMED / PENDING / CANCELLED).